■■ DETECTIVE DE ESTAFAS DIGITALES

Guion de Defensa Oral y Estructura de la Presentación FINAL

Diplomatura en Inteligencia Artificial — Exposición Académica

■ **Pautas para la Exposición Oral (5 a 7 minutos):**

• **Tono:** Profesional, natural, fluido y técnicamente riguroso sin tecnicismos innecesarios.

• **Idea Central a Transmitir:** *«Nuestro agente utiliza inteligencia artificial para comprender la consulta del usuario y decidir cuándo activar la Skill. Después, el motor Python analiza el mensaje mediante reglas verificables y devuelve un informe que el agente utiliza para explicar los riesgos en lenguaje sencillo.»*

• **Estructura:** Exposición de diapositivas (5-7 min), demostración de comandos en vivo (2 min) y preguntas del profesor.

DIAPOSITIVA 1: Portada e Introducción al Proyecto

■ **Contenido visual proyectado:**

- **Título:** Detective de Estafas Digitales — Skill de IA Segura, Modular y Auditable.
- **Contexto académico:** Diplomatura en Inteligencia Artificial.
- **Eje central:** Agente Asistente • Análisis Determinístico • Mitigación de Inyección • Mínimo Privilegio.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Buenas tardes profesor y compañeros. Hoy les presentamos 'Detective de Estafas Digitales', un proyecto desarrollado como trabajo práctico final de la Diplomatura en Inteligencia Artificial.

El objetivo central fue construir una Skill real y auditable para agentes de IA que ayude a cualquier usuario a evaluar mensajes sospechosos de WhatsApp, SMS o correo, diseñada desde su arquitectura bajo principios estrictos de ciberseguridad: tratamiento de mensajes como datos no confiables, análisis estático sin salida a red y separación estricta entre la inteligencia del agente y la lógica de inspección.»

DIAPOSITIVA 2: El Problema: Ciberestafas y Manipulación Psicológica

■ **Contenido visual proyectado:**

• **Amenazas cotidianas:** Phishing y smishing masivo en Argentina (suplantación de Mercado Pago, Cuenta DNI, bancos y ANSES).

• **Ingeniería social:** Manipulación por miedo al bloqueo de cuentas, urgencias artificiales o falsos premios con DEBIN.

• **El desafío para los agentes de IA:** Analizar mensajes hostiles sin que el modelo sea manipulado por Prompt Injection ni forzado a visitar enlaces maliciosos.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Todos los días vemos a personas caer en trampas digitales porque los estafadores apelan a la urgencia o al miedo de perder su cuenta o sus fondos. Pero cuando intentamos que un asistente de inteligencia artificial ayude en esta tarea, nos encontramos con un dilema de seguridad: el mensaje que analizamos puede contener trampas deliberadas destinadas a engañar al propio modelo de lenguaje.

Además, una herramienta de asistencia jamás debería visitar enlaces desconocidos ni inventar probabilidades estadísticas sin sustento verificable.»

DIAPOSITIVA 3: Objetivo General y Alcance del MVP

■ **Contenido visual proyectado:**

• **Lo que SÍ hace el MVP:** Análisis estático de texto y enlaces, detección de patrones conocidos de fraude, mitigación activa de Prompt Injection, clasificación de riesgo explicable (BAJO/MEDIO/ALTO/INDET) y doble reporte (JSON formal + Markdown accesible).

• **Restricciones deliberadas:** Zero Network (cero peticiones HTTP/DNS), sin ejecución dinámica (prohibido eval/exec), sin dependencias externas para el motor funcional (100% biblioteca estándar de Python).

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Para resolver este problema definimos un alcance muy claro: una herramienta determinística, local y explicable, sin dependencias externas en producción.

Excluimos deliberadamente cualquier conexión a internet: si una herramienta analiza un link visitándolo, revela la dirección IP de la víctima y expone al sistema a exploits. Nuestro análisis es 100% estático, inspeccionando la sintaxis del mensaje y las URLs sin interactuar con los servidores atacantes.»

DIAPOSITIVA 4: Arquitectura General: Dónde Interviene la IA

■ **Contenido visual proyectado:**

• **Capa Cognitiva (LLM + Agente Antigravity):** Interpreta la intención del usuario en lenguaje natural, descubre la Skill mediante SKILL.md y traduce los resultados técnicos a recomendaciones empáticas.

• **Capa Determinística (Motor Python):** Analiza patrones léxicos, sintaxis de URLs y calcula un puntaje heurístico reproducible sin riesgo de alucinación.

• **Flujo de ejecución:** Usuario → Agente IA → SKILL.md → Motor Python (worker.py vía STDIN) → Reporte estructurado (JSON) → Agente IA → Explicación final al usuario.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Acá llegamos al punto neurálgico del proyecto para la Diplomatura: ¿dónde interviene la Inteligencia Artificial?

Nuestro agente utiliza inteligencia artificial para comprender la consulta del usuario en lenguaje natural y decidir cuándo activar la Skill según su manifiesto SKILL.md. Después, el motor Python analiza el mensaje mediante reglas verificables y devuelve un informe estructurado que el agente vuelve a interpretar para explicar los riesgos en lenguaje sencillo.

No reemplazamos la IA por scripts, ni dejamos que la IA alucine indicadores: combinamos la capacidad lingüística del LLM con la precisión auditable de una herramienta determinística.»

DIAPOSITIVA 5: El Contrato del Agente: SKILL.md

■ **Contenido visual proyectado:**

• **Descubrimiento progresivo:** Encabezado YAML estándar (name, description) para que el agente reconozca cuándo activarla.

• **Tratamiento de datos no confiables:** El texto analizado se encapsula como 'Untrusted Data', nunca como instrucción operativa.

• **Comunicación robusta:** Se recomienda la invocación mediante STDIN para evitar interpolaciones de shell y fallas por comillas.

• **Mitigación de Prompt Injection:** Defensa en profundidad a nivel de directivas del agente y filtros de reglas en Python.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«El archivo SKILL.md funciona como el contrato operativo del agente. No solo le enseña en qué momento intervenir cuando el usuario le pega un mensaje sospechoso, sino que fija reglas de seguridad fundamentales:

Primero, establece que todo contenido entrante es 'Untrusted Data' (información no confiable), lo que previene que el modelo obedezca órdenes maliciosas camufladas en el texto. Y segundo, prescribe la invocación mediante STDIN, lo que evita que comillas dobles o caracteres de shell rompan la ejecución o generen vulnerabilidades de inyección.»

DIAPOSITIVA 6: El Motor Funcional: Pipeline Python

■ **Contenido visual proyectado:**

• **worker.py:** Punto de entrada seguro; valida límites de entrada (máx. 15.000 chars), previene Path Traversal y lee vía STDIN o archivo.

• **analyzer.py:** Detección de coerción, urgencia, bloqueo, premios y patrones de inyección de instrucciones.

• **url_analyzer.py:** Extracción con Regex y auditoría sintáctica offline (IPs numéricas, subdominios falsos, homóglifos Punycode).

• **risk_classifier.py:** Cálculo de puntaje heurístico de severidad acumulada (0-100) y categoría de riesgo.

• **report_generator.py:** Generación de reportes duales con advertencia explícita sobre el significado del puntaje.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«En el motor funcional, cada módulo cumple una función técnica bien delimitada, sin dependencias de terceros:

worker.py actúa como aduana de seguridad validando longitudes y rutas seguras; analyzer.py busca disparadores psicológicos de fraude e intentos de inyección; url_analyzer.py descompone cada enlace para detectar engaños sintácticos; risk_classifier.py asigna pesos transparentes; y report_generator.py emite la respuesta estructurada.

Es importante destacar que estos módulos son herramientas determinísticas al servicio del agente, no modelos de IA en sí mismos.»

DIAPOSITIVA 7: Auditoría Estática de Enlaces (Zero Network)

■ **Contenido visual proyectado:**

• **Indicadores estructurales:** Direcciones IP literales (http://192.168.1.50/login), subdominios de suplantación (banco.galicia.com.dominio-trucho.xyz), homóglifos/punycode (xn--...), credenciales en URL y TLDs sospechosos (.xyz, .top, .online).

• **Aclaración educativa clave:** 'HTTPS NO significa seguro'. HTTPS solo cifra la conexión entre el usuario y el servidor. Un delincuente puede tramitar un certificado SSL gratuito en minutos; la comunicación viajará cifrada, pero irá directo al estafador.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Una de las fortalezas más prácticas del proyecto es la auditoría estática de enlaces. Detectamos tácticas comunes como subdominios truchos o caracteres rusos disfrazados de letras latinas, todo de manera offline.

Y además, incluimos un concepto educativo indispensable: 'HTTPS no significa seguro'. Muchos usuarios creen que el candado verde garantiza que el sitio es legítimo. Les explicamos que HTTPS solo cifra el canal; si el sitio pertenece a un estafador, las contraseñas viajarán cifradas, pero directamente a las manos del delincuente.»

DIAPOSITIVA 8: Seguridad y Niveles de Aislamiento

■ **Contenido visual proyectado:**

• **Restricciones en Código Python:** Path Traversal neutralizado (Path.resolve() limitado a carpeta segura), bloqueo estricto de secretos (.env, id_rsa), sanitización de caracteres y prohibición absoluta de eval/exec.

• **Aislamiento en Contenedor (Dockerfile implementado):** Imagen ligera python:3.13-slim, usuario no privilegiado (UID 10001), sistema de archivos de solo lectura (--read-only) y bloqueo total de red (--network none).

• **Honestidad técnica:** Docker aísla recursos del sistema operativo, pero no elimina todos los riesgos; la seguridad es multicapa.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«En ciberseguridad no existen las soluciones mágicas ni la 'inmunidad absoluta'. Por eso aplicamos una estrategia de defensa en profundidad.

En el código Python prevenimos desbordamientos, bloqueamos el acceso a archivos del sistema y filtramos secretos. Y a nivel de infraestructura, implementamos un Dockerfile específico para empaquetar el motor con un usuario sin privilegios, sistema de archivos de solo lectura y el flag --network none, que garantiza a nivel del kernel de Linux que el contenedor no pueda abrir ningún socket a internet.»

DIAPOSITIVA 9: Batería de Pruebas Automatizadas con Pytest

■ **Contenido visual proyectado:**

• **Total ejecutado:** 36 pruebas automatizadas aprobadas en 1.05 segundos (100% de éxito).

• **Casos cubiertos:** Invocación real de CLI y STDIN, manejo de comillas y caracteres Unicode, rechazo de Path Traversal, entradas excesivas, mitigación de Prompt Injection, auditoría de URLs y flujo de integración Agente-Skill.

• **Transparencia:** Pasar el 100% de los tests no equivale a un 100% de cobertura del código ni garantiza detectar estafas no catalogadas.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Para validar el proyecto no nos quedamos en pruebas manuales. Diseñamos una suite automatizada con pytest que creció de 27 a 36 pruebas unitarias y de integración, todas ejecutadas con éxito en apenas un segundo.

Probamos la entrada por STDIN con textos de WhatsApp llenos de tildes y comillas, intentos de inyección de comandos, ataques de desbordamiento de caracteres y simulaciones completas del ciclo de vida del agente. Esto nos da certeza matemática de que las correcciones funcionan y no rompieron nada anterior.»

DIAPOSITIVA 10: Demostración Funcional y Casos Reales

■ **Contenido visual proyectado:**

• **Caso 1 (Phishing Bancario):** Alerta de bloqueo + enlace engañoso ➔ Riesgo ALTO (100/100).

• **Caso 2 (Prompt Injection):** Intento de cambiar el rol del agente ➔ Detectado y contenido.

• **Caso 3 (Mensaje Legítimo):** Conversación habitual ➔ Riesgo BAJO (0/100).

• **Aclaración imprescindible:** El resultado es un puntaje heurístico de riesgo (0-100), NO una probabilidad estadística. 100/100 significa acumulación de indicadores graves; 0/100 significa que no se detectaron patrones conocidos, sin que ello constituya un certificado absoluto de legitimidad.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«En la demostración práctica mostramos tres casos emblemáticos:

El primero es un phishing bancario típico que acumula urgencia y un enlace falso, alcanzando un riesgo ALTO de 100/100. El segundo es un mensaje que incluye una inyección de instrucciones que intenta obligar al sistema a ignorar sus directivas, el cual es neutralizado y catalogado como amenaza. Y el tercero es un mensaje normal que arroja riesgo BAJO con 0/100.

Queremos remarcar algo fundamental: 100/100 no es una probabilidad del 100% de estafa; es un puntaje heurístico basado en reglas. Del mismo modo, un 0/100 no garantiza al 100% que el remitente sea honesto; simplemente indica que no encontramos indicadores conocidos.»

DIAPOSITIVA 11: Honestidad Académica: Limitaciones del Sistema

■ **Contenido visual proyectado:**

• **Reglas heurísticas vs. Modelo propio:** El motor utiliza expresiones regulares y umbrales fijos; no fue entrenado con aprendizaje automático supervisado.

• **Falsos positivos y negativos:** Nuevas variantes de estafas que no utilicen las palabras clave o enlaces engañosos conocidos pueden no ser detectadas.

• **Análisis offline:** Al no resolver enlaces acortados (bit.ly) por seguridad, no es posible verificar la URL final de destino.

• **Formato de entrada:** Analiza únicamente texto plano; no procesa imágenes (OCR), archivos adjuntos PDF ni ejecutables APK.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Siguiendo las mejores prácticas académicas, reconocemos con claridad las limitaciones del proyecto:

El motor se basa en heurísticas expertas, no en una red neuronal entrenada con un dataset etiquetado. Esto significa que si un ciberdelincuente redacta una estafa con un vocabulario completamente novedoso o utiliza un enlace acortado que nosotros no abrimos por seguridad, el sistema puede arrojar un falso negativo.

Tampoco procesamos capturas de pantalla ni archivos APK, ya que el alcance se limitó a texto plano.»

DIAPOSITIVA 12: Conclusiones y Aprendizajes

■ **Contenido visual proyectado:**

• **Aporte principal:** Demostración práctica de cómo integrar un LLM como interfaz cognitiva con un motor local determinístico y seguro.

• **Ciberseguridad aplicada:** Implementación real de mínimos privilegios, sanitización de entradas y defensa contra manipulaciones por inyección.

• **Lección de la Diplomatura:** Para construir agentes inteligentes verdaderamente útiles y confiables, la seguridad y la explicabilidad deben diseñarse desde el primer día.

■■ GUION DEL ORADOR (Qué decir ante el profesor):

«Para cerrar: este trabajo práctico nos permitió aplicar de forma concreta los temas de la Diplomatura. Aprendimos que la mejor arquitectura para un agente no consiste en delegar todo ciegamente a un modelo de lenguaje, sino en combinar su capacidad de comprensión con herramientas locales auditables, determinísticas y seguras.

Logramos un sistema que protege al usuario final de las estafas y, al mismo tiempo, se protege a sí mismo de las amenazas digitales. Muchas gracias y quedo a disposición del profesor para la demostración y sus preguntas.»

■ Chuleta de Comandos Rápidos para la Demo en Vivo

Comandos recomendados en PowerShell para la demostración en vivo frente al tribunal:

Caso a Demostrar	Comando exacto a ejecutar	Aspecto clave a resaltar
1. Demo Phishing Bancario	`Get-Content examples/phishing_bancario.txt | python scripts/worker.py --stdin`	Riesgo ALTO (100/100). Detecta subdominio falso y urgencia de bloqueo.
2. Demo Prompt Injection	`Get-Content examples/prompt_injection.txt | python scripts/worker.py --stdin`	Detecta inyección de instrucciones y la neutraliza sin ejecutar comandos.
3. Demo Mensaje Legítimo	`Get-Content examples/mensaje_legitimo.txt | python scripts/worker.py --stdin`	Riesgo BAJO (0/100) con advertencia de que la ausencia de señales no es garantía.
4. Salida en formato JSON	`Get-Content examples/phishing_bancario.txt | python scripts/worker.py --stdin --json`	Genera JSON conforme con schema oficial y score_type 'heuristic_severity_score'.
5. Batería de Tests Pytest	`.venv\Scripts\python.exe -m pytest -v`	Ejecuta las 36 pruebas automatizadas aprobadas en ~1.05s.

■ Respuestas Preparadas a las 10 Preguntas Clave del Profesor

Respuestas breves, directas y rigurosas a las preguntas más probables del tribunal:

1. ¿Dónde está la inteligencia artificial?

La IA reside en el Modelo de Lenguaje (LLM) operado por el Agente de Antigravity. El agente interpreta la consulta del usuario en lenguaje natural, comprende cuándo debe activar la Skill a partir de las instrucciones de SKILL.md, invoca el motor seguro y luego utiliza el informe estructurado devuelto para redactar una explicación empática y comprensible. El motor Python actúa como una herramienta determinística especializada al servicio de la IA.

2. ¿Por qué no entrenaron una red neuronal?

Porque para la auditoría de seguridad y la detección de estafas críticas se requiere explicabilidad 100% auditable y determinismo sin riesgo de alucinación. Si un modelo neuronal emite una probabilidad difusa, no podemos verificar el motivo exacto ni garantizar que una inyección de texto no distorsione el resultado. Las reglas heurísticas y el análisis sintáctico garantizan que la misma entrada produzca siempre exactamente la misma evaluación verificable.

3. ¿Qué diferencia hay entre un agente y una Skill?

El Agente es la entidad con capacidad de razonamiento basada en un LLM que interactúa con el usuario, toma decisiones y planifica acciones. La Skill es un paquete modular de capacidades (manifiesto SKILL.md, herramientas ejecutables y referencias) que amplía las facultades del agente para resolver una tarea específica con límites bien definidos.

4. ¿Por qué utilizaron Python y reglas?

Python es el estándar de facto en integración de agentes y ciencia de datos. Utilizamos la biblioteca estándar de Python (re, urllib.parse, argparse, json) porque permite ejecutar el motor sin dependencias de terceros, con tiempos de respuesta de milisegundos, consumo mínimo de memoria y una superficie de ataque reducida a cero vulnerabilidades de paquetes externos.

5. ¿Qué significa una puntuación de 100/100?

Significa que el mensaje acumuló suficientes indicadores de severidad grave (por ejemplo, amenazas de bloqueo + solicitud de claves + enlaces engañosos) para alcanzar el tope de la escala heurística. NO significa que exista una probabilidad matemática del 100% de que sea un fraude. De forma análoga, un 0/100 significa que no se detectaron patrones conocidos, no que el mensaje sea garantizadamente legítimo.

6. ¿Cómo evitan que el agente obedezca instrucciones maliciosas?

Mediante una estrategia de defensa en profundidad: en SKILL.md se instruye formalmente al agente a tratar todo mensaje del usuario como 'Untrusted Data' (datos no confiables). En el motor Python, el texto no se ejecuta jamás en shells ni con eval(), sino que se procesa como texto pasivo en memoria. Además, el módulo analyzer.py detecta patrones de inyección ('ignore previous instructions', etc.) y alerta inmediatamente de la amenaza.

7. ¿Qué ocurre si el mensaje parece legítimo, pero en realidad es una estafa?

El sistema arrojará un puntaje bajo o indeterminado, pero siempre emitirá una advertencia preventiva obligatoria: la ausencia de indicadores conocidos no es garantía absoluta de seguridad. Se recomienda al usuario verificar directamente por canales oficiales sin ingresar datos personales.

8. ¿Por qué no verifican enlaces en Internet?

Por la estricta política Zero Network adoptada en el proyecto. Visitar un enlace malicioso activo revela la dirección IP de la víctima al estafador, confirma que el número de teléfono o correo es válido y expone el sistema a posibles ataques o descargas automáticas de malware (drive-by downloads). Nuestro análisis es puramente estático y local.

9. ¿Qué medidas de seguridad están efectivamente implementadas?

En el código Python están verificadas: prevención de Path Traversal, bloqueo de archivos secretos (.env, id_rsa), límites estrictos de entrada (15.000 caracteres), entrada por STDIN sin interpolación de shell y ausencia de ejecución dinámica. A nivel de contenedor, se implementó el Dockerfile con usuario sin privilegios, sistema de archivos de solo lectura y aislamiento de red (--network none).

10. ¿Cómo demostraron que la Skill funciona dentro de Antigravity?

La Skill se instaló en el repositorio estándar de Antigravity (.agents/skills/detective-estafas-digitales/SKILL.md). El agente lee el archivo de metadatos, reconoce las intenciones del usuario en lenguaje natural y sabe invocar el script scripts/worker.py vía STDIN. Verificamos tanto la invocación CLI con subprocesos como la integración lógica en las pruebas automatizadas.